

Week 09

Advanced and Practical UVM

Jesus Sanchez

Dept. d'Arquitectura de Computadors
Universitat Politècnica de Catalunya

Processor Verification

Master in Innovation and Research in Informatics

Objectives of the Week

- Move from “toy” UVM examples to scalable, configurable environments
- Learn UVM service mechanisms (factory, config, report)
- Understand communication via TLM (analysis ports/FIFOs)
- Learn how to handle multiple agents and coordinate via virtual sequencers

A Complete UVM Skeleton

```
package coffee_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

// Enumerations
// -----
typedef enum logic {
    SMALL = 1'b0;
    LARGE = 1'b1;
} size_e;

typedef enum logic [1:0] {
    NONE
    EXPRESSO
    LATTE
    CAPPUCCINO = 2'd3;
} coffee_type_e;

// Transaction
// -----
class coffee_transaction extends uvm_sequence_item;
    // new
endclass;

// Sequencer
// -----
class coffee_sequencer extends uvm_sequencer#(coffee_transaction);
    `uvm_component_utils(coffee_sequencer)
endclass

// Driver
// -----
class coffee_driver extends uvm_driver#(coffee_transaction);
    `uvm_component_utils(coffee_driver)
    // new, build_phase, run_phase
endclass

// Monitor
// -----
```

Get the virtual interface from uvm_config_db

```
class coffee_monitor extends uvm_monitor;
    `uvm_component_utils(coffee_monitor)
    // new, build_phase, run_phase
endclass

// Agent
// -----
class coffee_agent extends uvm_agent;
    `uvm_component_utils(coffee_agent)
    // new, build_phase, connect_phase
endclass

// Scoreboard
// -----
class coffee_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(coffee_scoreboard)
    // new, write
endclass

// Environment
// -----
class coffee_env extends uvm_env;
    `uvm_component_utils(coffee_env)
    // new, build_phase, connect_phase
endclass

// Sequence
// -----
class coffee_sequence extends uvm_sequence#(coffee_transaction);
    `uvm_object_utils(coffee_sequence)
    // body
endclass

// Test
// -----
class coffee_test extends uvm_test;
    `uvm_component_utils(coffee_test)
    // new, build_phase, run_phase
endclass

endpackage : coffee_pkg
```

Create sequencer, driver, and monitor

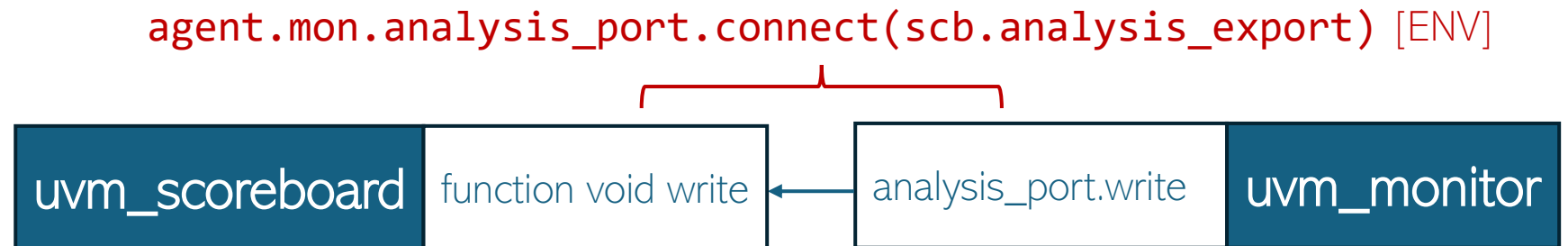
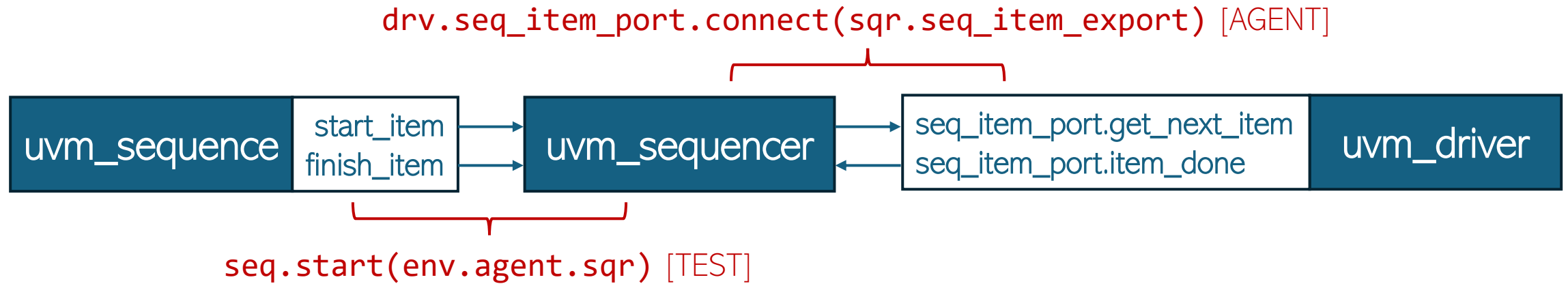
Connect driver, and sequencer

Create agent, and scoreboard

Connect monitor, and scoreboard

Create environment, and sequence

Communication and Connect Phases



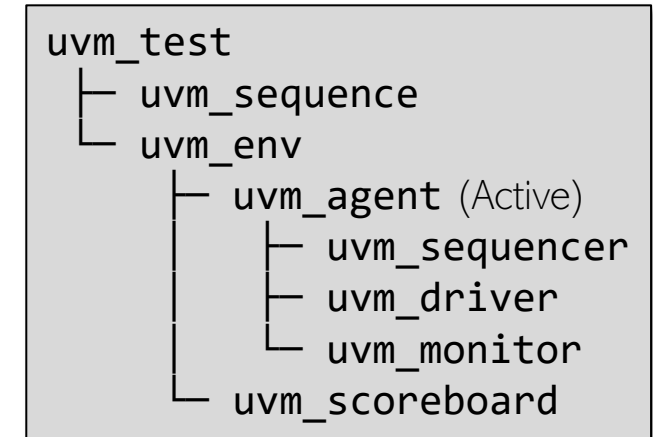
Agenda

1. Motivation for advanced features
2. Factory: creating and overriding components
3. Configuration mechanism
4. Reporting system
5. TLM, scoreboard, and coverage
6. Complex testbenches
7. Wrap-up and self-learning

Recap: Minimal UVM Example

■ What we built last week:

- A simple FIFO verification environment using UVM
- Included:
 - One `uvm_agent` (active) containing `sequencer`, `driver`, and `monitor`
 - A `uvm_env` connecting agent, `scoreboard`, and DUT interface
 - A `uvm_test` starting a single sequence



■ Key characteristics:

- ✓ Clear modular structure (driver, monitor, scoreboard)
- ✗ Static configuration: all components instantiated manually
- ✗ Hardcoded behavior: one sequence, one agent, fixed parameters
- ✗ Limited reuse: adapting to new DUTs or tests requires editing code

We verified a simple DUT successfully,
but real projects need scalability, configurability, and reuse

Why We Need More

- As designs grow ...
 - More interfaces, agents, and test scenarios appear
 - Verification teams must **reuse**, **reconfigure**, and **combine** environments easily
 - Manual edits to the testbench for each new DUT are **error-prone** and **not scalable**
- Challenges of larger projects: UVM service mechanisms

Problem	Example	Needed UVM Feature
Reuse	Same agent used in 3 environments	<code>uvm_factory</code>
Configuration	Change parameters or connections from test	<code>uvm_config_db</code>
Debug and control	Adjust verbosity, errors, runtime info	<code>uvm_report_server</code>
Communication	Connect components flexibly	TLM
Scalability	Multiple agents, sequences, VIPs	Virtual sequencers, hierarchical envs

UVM Service Mechanisms Overview

uvm factory

- Enables dynamic, name-based instantiation of components and objects
- Supports factory overrides for testbench reuse and variation

uvm config db

- Centralized configuration database for passing values across hierarchy
- Used for virtual interfaces, knobs, and handles

uvm report server

- Manages logging, verbosity, and message formatting
- Customizable to redirect output or suppress messages

uvm root

- Singleton that coordinates phases, components, and simulation flow

Agenda

1. Motivation for advanced features
2. Factory: creating and overriding components
3. Configuration mechanism
4. Reporting system
5. TLM, scoreboard, and coverage
6. Complex testbenches
7. Wrap-up and self-learning

What is the Factory?

The factory lets you choose which class to use at runtime — not hardcoded

■ Key points:

- Central registry for creating **components and objects** by type name
- Enables **flexible substitution** without modifying source code
- Supports **testbench reuse** and **late binding** of implementations



Factory Registration

Components
uvm_test
uvm_env
uvm_driver
uvm_monitor
uvm_sequencer

Objects
uvm_sequence
uvm_sequence_item
config objects

- Key points:
 - Every component or object must be registered to be created via the factory
- Use macros:
 - `uvm_component_utils(CLASS_NAME)` for components
 - `uvm_object_utils(CLASS_NAME)` for sequence items and other objects
- Code example:

```
class fifo_monitor extends uvm_monitor;  
  `uvm_component_utils(fifo_monitor)  
  ...  
endclass
```

```
class fifo_sequence extends uvm_sequence #(fifo_item);  
  `uvm_object_utils(fifo_sequence)  
  ...  
endclass
```

Registration tells the factory: “This class exists — you can create it.

Creating Components Via Factory

- Key points:

- Use `type_id::create()` instead of `new`
- Keeps code decoupled and override-friendly

- Code example:

```
drv = fifo_driver::type_id::create("drv", this);
```

- Benefits:

- Supports future overrides
- Works with configuration and introspection tools

Use `create()` to let the factory decide which class to instantiate

Overrides Example

■ Key points:

- Replace one class with another without editing the environment
- Two types:
 - Type override → applies globally
 - Instance override → applies to specific paths

■ Code example:

```
uvm_factory::get().set_type_override_by_name("my_driver", "my_driver_cov");  
uvm_factory::get().set_inst_override_by_name("env.agent.drv", "my_driver_cov");
```

Let you change behavior from the test, no need to touch the environment

Example of Usage

```
class packet extends uvm_sequence_item;
  rand bit [5:0] length;
  `uvm_object_utils(packet)
  constraint length_constr { length inside {[0:64]}; }
endclass
```

```
class short_packet extends packet;
  `uvm_object_utils(short_packet)
  constraint short_length_constr { length < 20; }
endclass
```

```
// In multiple locations throughout the testbench
packet my_packet = packet::type_id::create ("packet", this);
```

```
// In my uvm_test extended class
set_type_override_by_type(packet::get_type(), short_packet::get_type());
```

How do I use **short_packet** without editing my code?

Factory Introspection

■ Key points:

- Use `uvm_factory::print()` to see:
 - Registered types
 - Active overrides
 - Instantiation history

■ Code example:

```
uvm_factory::get().print();
```

■ Use case:

- Check if your override worked
- Debug unexpected instantiations

Instance Name	Type Name	Override Type
env	fifo_env	-
env.agent	fifo_agent	-
env.agent.seqr	fifo_sequencer	-
env.agent.drv	fifo_driver_cov	INST
env.agent.mon	fifo_monitor	-
env.scoreboard	fifo_scoreboard	-
env.coverage	fifo_coverage_collector	-
test	fifo_test_cov	TYPE

Factory Overrides:

TYPE OVERRIDE:

fifo_test → fifo_test_cov

INSTANCE OVERRIDE:

env.agent.drv → fifo_driver_cov

Agenda

1. Motivation for advanced features
2. Factory: creating and overriding components
3. Configuration mechanism
4. Reporting system
5. TLM, scoreboard, and coverage
6. Complex testbenches
7. Wrap-up and self-learning

Why Configuration is Needed

Configuration lets you control behavior without editing internal components

■ Key points:

- Avoid **hardcoded values** inside components (e.g., timeouts, addresses, modes)
- Enable **top-level control** from the test or environment
- Promote **reuse** and **scalability** across different test scenarios
- Decouple **test intent** from **component implementation**

The uvm_config_db API

■ Key points:

- **set()** is called in a higher-level component (e.g., test or env)
- **get()** is called in a lower-level component (e.g., driver or monitor)
- Typically used in the build_phase
- Works with any data type (int, string, **virtual interface**, etc.)

■ Code skeleton:

```
// In test or env
uvm_config_db#(T)::set(this, "path.to.component", "key", value);

// In component
uvm_config_db#(T)::get(this, "", "key", local_var);
```

= myself

Set from the top, get from the bottom — during build_phase

Examples

```
// In test
uvm_config_db#(int)::set(this, "env.agent*", "timeout", 100);
```

Matches all agents (wildcard)

```
// In driver or monitor
```

```
int timeout;
if (!uvm_config_db#(int)::get(this, "", "timeout", timeout))
    `uvm_fatal("NOVALUE", "Timeout not set")
```

```
// In test
```

```
uvm_config_db#(virtual fifo_if)::set(this, "env.agent.driver", "vif", vif);
```

```
// In driver
```

```
virtual fifo_if vif;
if (!uvm_config_db#(virtual fifo_if)::get(this, "", "vif", vif))
    `uvm_fatal("NOVIF", "Virtual interface not set")
```

Use set() to configure behavior — get() to retrieve it safely

Hierarchical Paths and Wildcards

■ Key points:

- Paths define **where** the value is set or retrieved
- Wildcards allow **flexible matching**:
 - "env.agent*" → all agents
 - "env.*.driver" → all drivers under env
 - "*" → global match (use with caution)

■ Common pitfall:

- **Wrong path = silent failure** (no error, just no value)
- Always double-check instance names and hierarchy

Paths must match the component hierarchy — wildcards help, but be precise

Best Practices

■ Tips:

- Always log config values after `get()`:
 ``uvm_info("CONFIG", $sformatf("Timeout = %0d", timeout), UVM_LOW)`
- Centralize **`set()`** calls in the test or top-level env
- Use wildcards for scalable configuration
- Check return value of **`get()`** — don't assume success
- Avoid setting config in `run_phase` — use `build_phase`

Treat configuration like a contract — set it clearly, get it safely, log it always

10:00

Agenda

1. Motivation for advanced features
2. Factory: creating and overriding components
3. Configuration mechanism
4. Reporting system
5. TLM, scoreboard, and coverage
6. Complex testbenches
7. Wrap-up and self-learning

Why Use UVM Reporting?

UVM reporting is your testbench's voice — structured, controllable, and customizable

■ Key points:

- Provides a **unified mechanism** for logging, messaging, and error handling
- Replaces ad-hoc **\$display**, **\$fatal**, etc. with structured macros
- Supports **verbosity control**, **message filtering**, and **custom formatting**
- Helps with **debugging**, **regression analysis**, and **testbench transparency**

Using UVM Reporting Macros

■ Key macros:

```
`uvm_info("DRV", "Transaction sent", UVM_MEDIUM)  
`uvm_warning("FIFO", "Almost full!")  
`uvm_error("SCORE", "Data mismatch")  
`uvm_fatal("TRANSACTION", "Wrong randomization")
```

← Can be filtered

← Simulation exists

■ Verbosity levels

Keyword	Level
UVM_NONE	Silent
UVM_LOW	Minimal
UVM_MEDIUM	Default
UVM_HIGH	Verbose

Use tags to identify message sources

Controlling Output with Report Server

- Key Points:

- Verbosity control from command line:

```
+uvm_set_verbosity=env.agent.drv,UVM_HIGH
```

- Can redirect output to files or custom handlers
- **uvm_report_server** manages:
 - Message formatting
 - Filtering
 - Destination (console, file, etc.)

You control what gets printed — globally or per component

Customizing the Report Server: An Example

```
class my_report_server extends uvm_report_server;

    // Override to control which messages are printed

    function bit is_enabled(int verbosity, uvm_severity severity, string id);
        // Always show errors and warnings
        if (severity == UVM_ERROR || severity == UVM_FATAL || severity == UVM_WARNING)
            return 1;

        // Suppress low-verbosity info messages unless from "SCORE"
        if (severity == UVM_INFO) begin
            if (verbosity < UVM_MEDIUM && id != "SCORE")
                return 0;
            end

        // Allow all other messages
        return 1;
    endfunction

endclass

// Activate the custom report server
initial begin
    uvm_report_server::set_server(new my_report_server());
end
```

```
`uvm_info("DRV", "Transaction sent", UVM_MEDIUM)
`uvm_warning("FIFO", "Almost full!")
`uvm_error("SCORE", "Data mismatch")
```

Agenda

1. Motivation for advanced features
2. Factory: creating and overriding components
3. Configuration mechanism
4. Reporting system
5. TLM, scoreboard, and coverage
6. Complex testbenches
7. Wrap-up and self-learning

Why TLM Instead of Mailboxes or Direct Calls?

- **Transaction-Level Modeling** provides a standardized communication mechanism
- Decouples producer (monitor) from consumer (scoreboard/coverage)
 - Avoids tight coupling and manual synchronization
- Supports multiple subscribers (one monitor can feed many consumers)
 - More scalable than mailboxes or direct method calls

TLM = flexible, reusable communication channel in UVM

Analysis Ports and Imps

■ Key Elements:

- `uvm_analysis_port` → producer side (monitor publishes transactions)
- `uvm_analysis_imp` → consumer side (scoreboard implements `write()`)
- `uvm_analysis_export` → intermediate connector (used in envs)

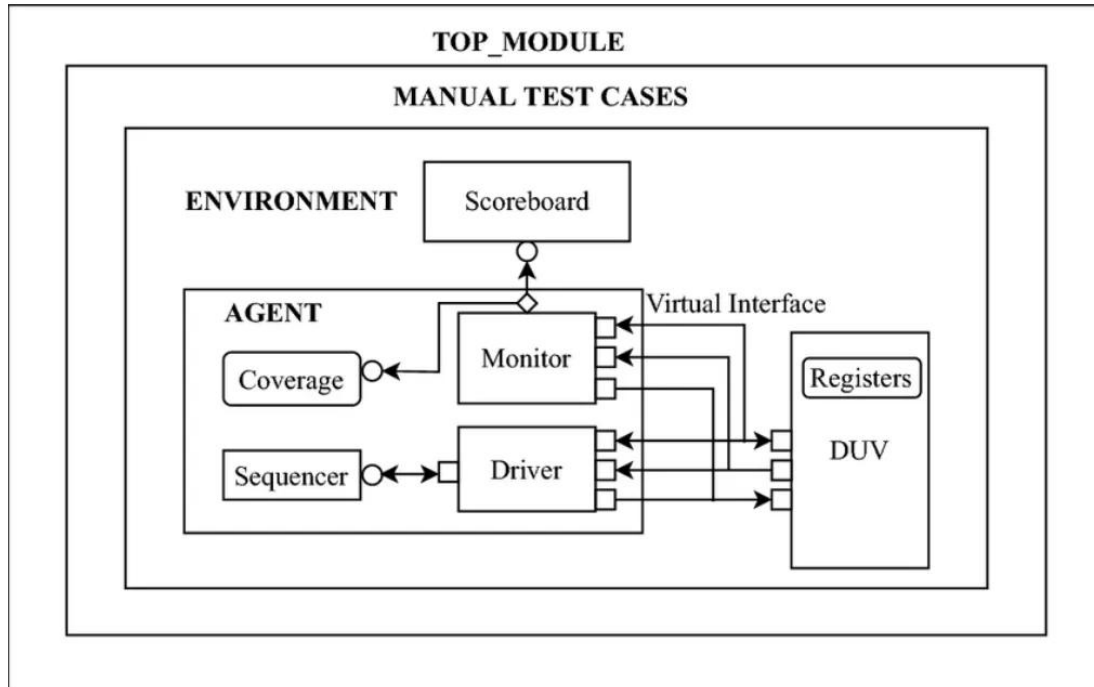
Monitor pushes transactions out, scoreboard pulls them in — via TLM ports

■ Wiring Producers and Consumers:

- Connections are made in `connect_phase`.
- Type checking is automatic (parameterized ports must match)

```
function void connect_phase(uvm_phase phase);  
    mon.ap.connect(sb.analysis_export);  
endfunction
```

And Last One ... UVM Subscriber



```
class fifo_cov_collector extends uvm_subscriber #(fifo_item);
    `uvm_component_utils(fifo_cov_collector)

    covergroup fifo_cov;
        coverpoint t.data;
        coverpoint t.addr;
        cross t.data, t.addr;
    endgroup

    function void build_phase(uvm_phase phase);
        fifo_cov = new();
    endfunction

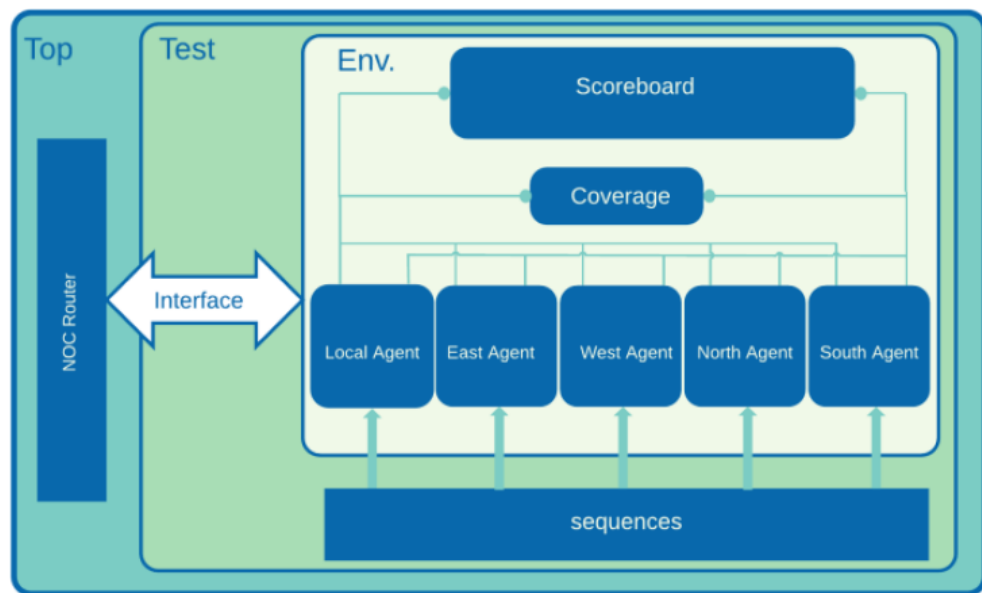
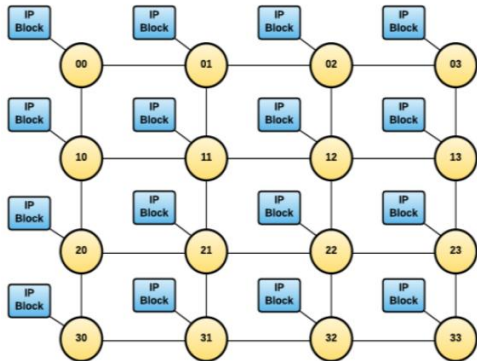
    // Called automatically when monitor publishes a transaction
    function void write(fifo_item t);
        fifo_cov.sample();
        `uvm_info("COV", $sformatf("Sampled data=%0d addr=%0d", t.data, t.addr),
        endfunction
    endclass
```

It cleanly separates checking (scoreboard) from measuring (coverage)

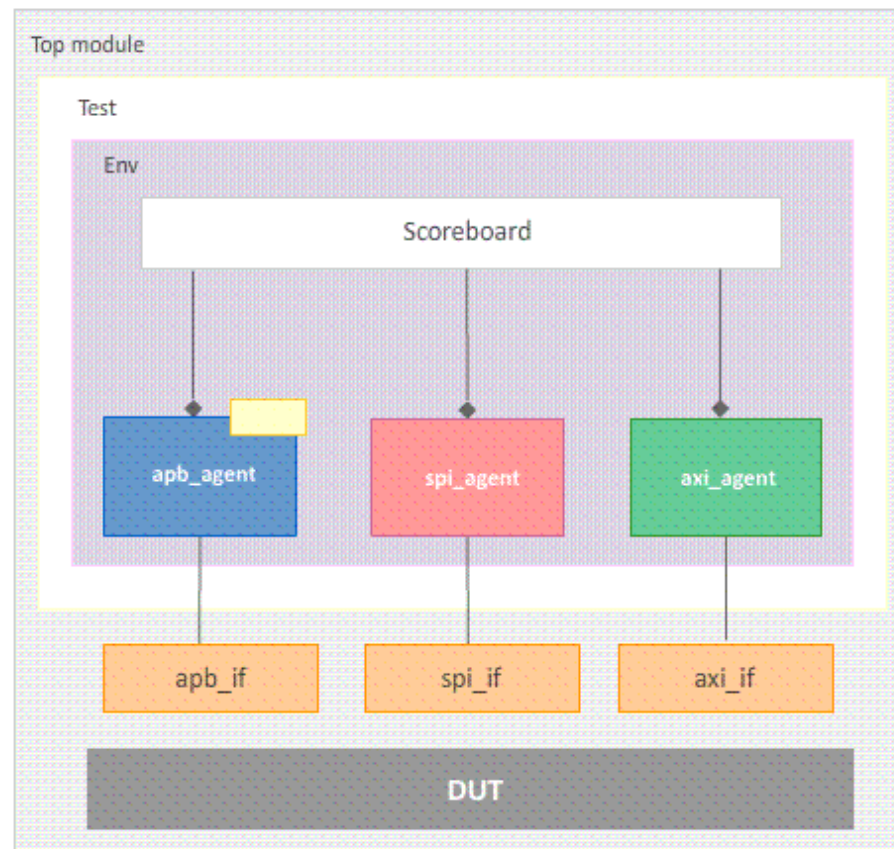
Agenda

1. Motivation for advanced features
2. Factory: creating and overriding components
3. Configuration mechanism
4. Reporting system
5. TLM, scoreboard, and coverage
6. Complex testbenches
7. Wrap-up and self-learning

Multiple Active Agents



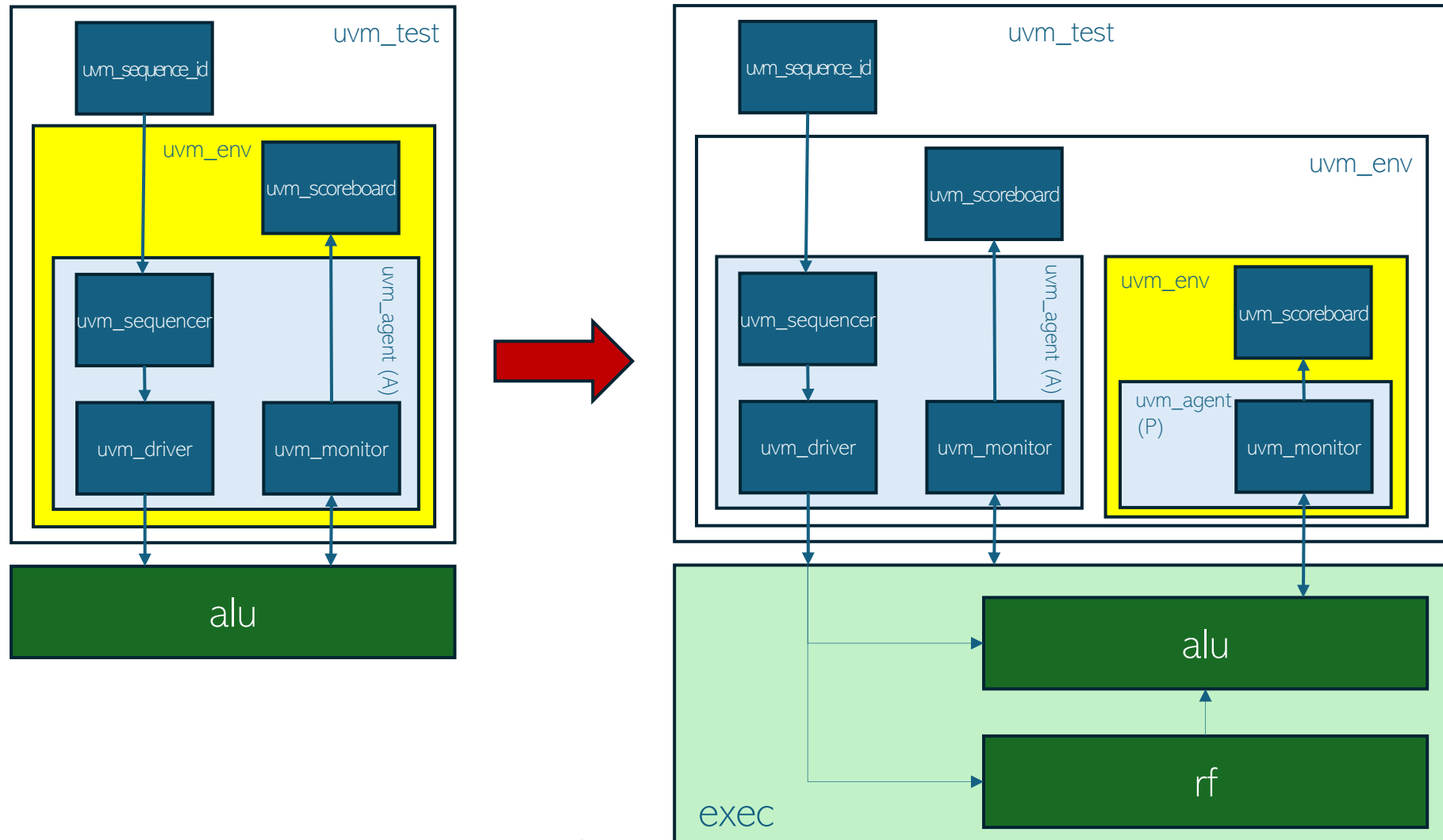
Mesh router with 4 ports



DUT with 3 buses

Source: [A Narrative of UVM Testbench Environment for Interconnection Routers](#)

Active and Passive Agents



Several Tests with Same Environment

```
class sanity_test extends uvm_test;
  `uvm_component_utils(sanity_test)

  my_env env;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = my_env::type_id::create("env", this);
  endfunction

  task run_phase(uvm_phase phase);
    phase.raise_objection(this);
    simple_seq seq = simple_seq::type_id::create("seq");
    seq.start(env.agent.seqr);
    phase.drop_objection(this);
  endtask
endclass
```

```
class stress_test extends uvm_test;
  `uvm_component_utils(stress_test)

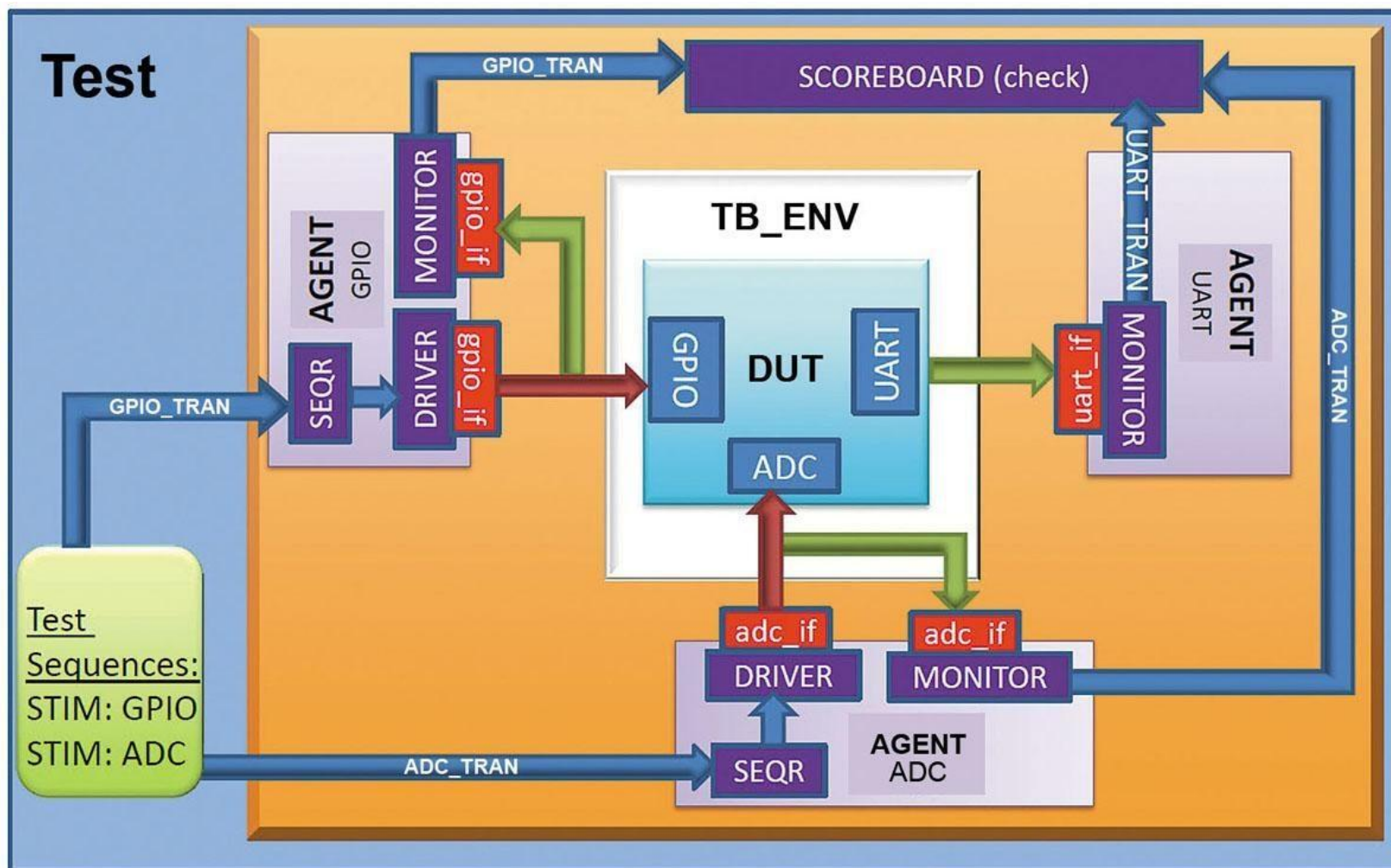
  my_env env;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = my_env::type_id::create("env", this);
  endfunction

  task run_phase(uvm_phase phase);
    phase.raise_objection(this);
    random_seq seq = random_seq::type_id::create("seq");
    seq.num_items = 1000; // more traffic
    seq.start(env.agent.seqr);
    phase.drop_objection(this);
  endtask
endclass
```

Just different sequences

Another Example ...



Agenda

1. Motivation for advanced features
2. Factory: creating and overriding components
3. Configuration mechanism
4. Reporting system
5. TLM, scoreboard, and coverage
6. Complex testbenches
7. Wrap-up and self-learning

Summary of Key Concepts from this Session

- UVM service mechanisms
 - Factory, configDB, and reporting
- TLM (analysis ports) for scoreboards and coverage
- More complex UVM architectures

Link to Next Week

- Week 10: Processor-Specific Verification Challenges
 - Split in blocks: bottom-up
 - Golden ISS models
 - Standard buses and VIPs

LECTURE SCHEDULE

1st hour: Lecture on Processor-Specific Verification
2nd hour: Q&A of the course (lectures and labs)

Self-Learning Corner

■ Videos

- [UVM Explained Through a Coffee Machine](#)
- [UVM Basics by Synopsis](#)

■ Articles

- [UVM Cookbook | Cookbook | Siemens Verification Academy](#) (also [here](#))

■ Presentations

- [Is it easy to get started with UVM, or should I use Formal instead?](#)

- Useful UVM macros: e.g. in `uvm_sequence`

```
task body;
    repeat (8)
    begin
        `uvm_do(req) ← start_item(req);
                        assert(tx.randomize());
                        finish_item(tx);
    end
endtask
```

■ Slides

- [UVM Basic by ARM](#)

“Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it.”

Brian W. Kernighan